Detecting Outdated & Vulnerable JavaScript Libraries

in CommonCrawl Web Data

Radboud University — Big Data (NWI-IBC036) — June 2026
s1122062

Key Finding
Out of 4,802,842 HTML pages crawled from the April 2021 CommonCrawl segment, 1,212,225 pages (25.2%) loaded at least one known-vulnerable JavaScript library. jQuery seemed to be dominating with over 1.4 million script-tag references, with version 1.12.4 alone appearing on more than 100,000 pages.

1. Motivation

JavaScript (JS) libraries like jQuery, Bootstrap, and AngularJS are used by the vast majority of websites on the web. One problem here is that the sites that use these libraries never update them. Old versions however are vulnerable, as there are publicly available vulnerability exploits against them (CVEs) that attackers actively exploit for XSS and more.

The project asks a simple but revealing question, namely, how many pages in a real web crawl are still loading a JavaScript library with a known vulnerability? The CommonCrawl dataset from April 2021 was provided, which is a monthly snapshot of a large portion of the web. The project framework incorporates a pipeline to scan JS script tags across approximately 5 million pages.

The idea addresses a real world concern at a large scale. While web security scanners like Snyk or Retire.js warn developers about this issue, the project tries to analyse this at crawl scale.

2. The Dataset

I used CommonCrawl CC-MAIN-2021-17, specifically the single-warc-segment made available on the Radboud redbad cluster's HDFS at /single-warc-segment/. This segment contains 640 WARC files, each approximately 1.2 GB, totalling around 768 GB of compressed web archive data.

For this project I processed 100 files (files 00000 through 00099), covering roughly 120 GB. The motivation here is that it is sufficient enough to observe large scale conclusions while remains somewhat computationally tractable (the 120 GB segment ran approximately 2 hours and 25 minutes). WARC (Web ARChive) files store the raw HTTP request/response pairs from the crawl, including full HTML bodies, headers, and metadata.

I filtered to HTTP responses with Content-Type text/html and a non-empty body, which gave me the 4.8 million HTML pages that form the basis of my analysis.

3. Approach & Pipeline

The pipeline ran on Spark 3.5.5 and was structured in five stages:

1. Load WARC files using the warc4spark data source (org.rubigdata.warc), parse HTTP headers, and filter to HTML responses only
2. Extract all <script src="..."> attributes from each HTML page using Jsoup into individual rows
3. Filter script URLs against a regex pattern matching known-vulnerable versions of jQuery, AngularJS, Bootstrap, Lodash, and Handlebars
4. Enrich each match with library name, version, TLD, domain, protocol, and a CVE identifier
5. Aggregate: breakdown by library/version, pages per TLD, top domains, CDN vs self-hosted, CVE frequency, and HTTP vs HTTPS

The vulnerability versions and CVEs:

Library	Vulnerable versions	Primary CVE
jQuery	1.x, 2.x, 3.0–3.4.x	CVE-2019-11358, CVE-2020-11022
AngularJS	1.x	CVE-2020-7676
Bootstrap	2.x, 3.0–3.3.x, 4.0–4.2.x	CVE-2019-8331
Lodash	< 4.17.21	CVE-2021-23337
Handlebars	< 4.7.7	CVE-2021-23369

4. Results

4.1 Overall exposure

The headline number is 25.2%. Namely, 25.2 percent of all HTML pages processed included at least one vulnerable JavaScript library. That is 1 out of every 4 pages on the web loading a dependency that is known to be exploitable.

Metric	Value
WARC files processed	100 (~120 GB)
Total HTML pages	4,802,842
Pages with vulnerable library	1,212,225
Fraction affected	25.2%
External CDN references	710,881
Self-hosted references	698,096

4.2 Library breakdown

jQuery is overwhelmingly dominant. Versions 1.x and 2.x account for the vast majority of vulnerable references, with 1.12.4 alone appearing 100,760 times. Bootstrap 3.3.7 is the most common non-jQuery vulnerable library at 31,221 occurrences. AngularJS 1.x, Lodash, and Handlebars are present but at much lower volumes.

Interestingly, jQuery 3.3.1 (85,974 occurrences) and 3.4.1 (61,776 occurrences) are in the top 10, whereby these are "modern" versions of jQuery that developers might consider safe, but both fall within the 3.0–3.4.x vulnerable range for CVE-2020-11022 (prototype pollution via jQuery.htmlPrefilter).

4.3 Geographic distribution (TLD)

.com domains dominate as expected, with 483,646 unique pages using a vulnerable jQuery. Russia (.ru) is second with 83,380, which is disproportionately high relative to global web share. Other notable entries are .jp (Japan, 42,493), .de (Germany, 39,584), .nl (Netherlands, 17,610). The .edu TLD stands out, where 18,542 pages on educational institutions are loading vulnerable jQuery, which is a really concerning finding given the sensitivity of academic data.

4.4 Top domains

The most-affected domain was simcast.com (432 pages with vulnerable jQuery), followed by www.163.com (China's largest news portal, 196 pages) and www.fandom.com (174 pages). Notable appearances include www.microsoft.com (125 pages), www.justice.gov (120 pages), and okra.stanford.edu (119 pages). Seeing Microsoft and the US Department of Justice in this list underlines just how pervasive the problem is, where even large, well-funded organisations were slow to update.

4.5 CDN vs self-hosted

The split between external CDN (710,881) and self-hosted (698,096) is nearly 50/50. This is an important nuance. CDN-loaded libraries are often pinned to a specific version and never updated, while self-hosted ones are baked into deployments that equally rarely see a refresh. Neither delivery method offers a clear advantage in practice.

4.6 CVE frequency

CVE-2019-11358 (jQuery 1.x prototype pollution) is the single most prevalent vulnerability with 835,277 occurrences. CVE-2020-11022 (jQuery 2.x/3.x htmlPrefilter XSS) follows with 422,745. Together, jQuery CVEs account for over 97% of all vulnerable references.

4.7 HTTP vs HTTPS

The vast majority of vulnerable-library pages are served over HTTPS (962,195 jQuery vs 321,858 HTTP). This tells an important story, where migrating to HTTPS, while important for transport security, did not motivate sites to update their JavaScript dependencies. The two concerns seem to be largely independent in practice.

5. The Process (and Everything that Went Wrong)

5.1 Getting started

I started in the Zeppelin notebook after reading the assignment. I played around with the cells provided and then started developing some new cells. This was after I had the security idea concerning JS libraries.

The first challenge was correctness of the vulnerability regex. Testing on local WARC files produced zero results. After investigation, the reason was a bit frustrating. Modern websites like debian.org and mediawiki.org bundle and hash their JavaScript, so there is no jquery-1.x.x.min.js in the HTML. The regex was correct though, so those sites simply do not use version-pinned CDN URLs.

I verified the correctness of the regex separately by running it against 4 known CDN URLs (e.g., code.jquery.com/jquery-1.12.4.min.js) and got 4/4 matches. This proved that my regex was correct.

5.2 Getting real data into the notebook

The big-data container cannot reach the redbad HDFS cluster (UnknownHostException on the namenode). So, I had to get a real WARC file into the container another way. The process:

1. In the redbad container, I ran hdfs dfs -get on one of the /10-warcs/ files to /tmp/, where I got a successful 1.2 GB download.
2. docker cp redbad:/tmp/ccwarc.warc.gz ... failed, because "copying between containers is not supported".
3. So, Copy redbad - Windows - big-data container. This worked, but the first attempt caught the file mid-download (107 MB instead of 1.2 GB), causing an EOFException in Spark. I genuinely did not know why this happened up until a certain point, where I realized, that my terminal is closed. I do not have I an idea when I closed it though. Then, I waited for the full download and repeated.

5.3 Scala/regex bugs

Several bugs appeared during writing my code:

- Invalid escape characters in regex (\d and \\d), as I used \d in the zpln, but in the Scala file I had to use \\d. This was a bit annoying as well as the program caught it at runtime.
- Regex only matched dash-style filenames (jquery-1.x) but not CDN path-style (jquery/1.x/jquery.min.js), which I fixed by changing [\-] to [/\-].
- Missing dot before .withColumn() call, which is a small and notable mistake but Scala gave an unhelpful "not found: value withColumn" error.

5.4 Cluster submission

Building the JAR with sbt assembly hit a Bouncy Castle deduplication conflict (two versions of the bcprov library pulled in transitively). Fixed it by adding an assemblyMergeStrategy to build.sbt that picks the first version and discards META-INF.

The first spark-submit with --deploy-mode cluster returned immediately with no output. I initially thought it had failed silently. It had actually submitted successfully, the yarn

application list revealed three jobs running under my student number, as I tried to submit more times thinking that the process failed silently. Two were killed and the remaining job ran overnight and completed successfully in approximately two and a half hours.

6. Conclusions

A quarter of the web was running a JavaScript library with a publicly known vulnerability in April 2021. jQuery 1.x, which has been EOL since 2016, was still the most common vulnerable dependency found, appearing on hundreds of thousands of pages. The near-equal CDN/self-hosted split suggests this is a maintenance culture problem, not a deployment method problem.

The geographic data shows this is global: .com, .ru, .jp, .de, .nl all have hundreds of thousands of affected pages. The presence of .edu, microsoft.com, and justice.gov in the data makes it personal. These sites are not just small forgotten.

HTTPS revealed an interesting finding, which would be contradictory at first, namely, the majority of vulnerable pages were served over HTTPS. Securing the transport layer and securing the application layer are independent concerns that organisations treat separately, and the application layer is losing.

7. What was Learned

Working at this scale forced me to think differently about vulnerabilities out there. Unit-testing a regex on 4 URLs is not enough obviously, and the patterns that look clean in the notebook turn out to have edge cases only visible at millions of rows. The pipeline design (filter early, cache middle way and avoid re-scanning the HTML twice) was as important as the logic itself.

I also learned that cluster mode spark-submit is not interactive in any meaningful way. The YARN web UI (redbad01.cs.ru.nl:8088) was inaccessible from my machines due to HSTS pinning and network restrictions so I relied entirely on yarn application -list and yarn logs to monitor and retrieve output. If the job had failed silently I would only have found out from the logs, which is a strong argument for writing output to HDFS rather than stdout in future work.

The biggest lesson regarding dependency conflicts (Bouncy Castle, warc4spark jitpack vs local JAR) is the invisible tax on Spark JAR builds. They do not cause compilation failures but they rather cause mysterious runtime behaviour that is hard to diagnose without knowing to look at the merge strategy.